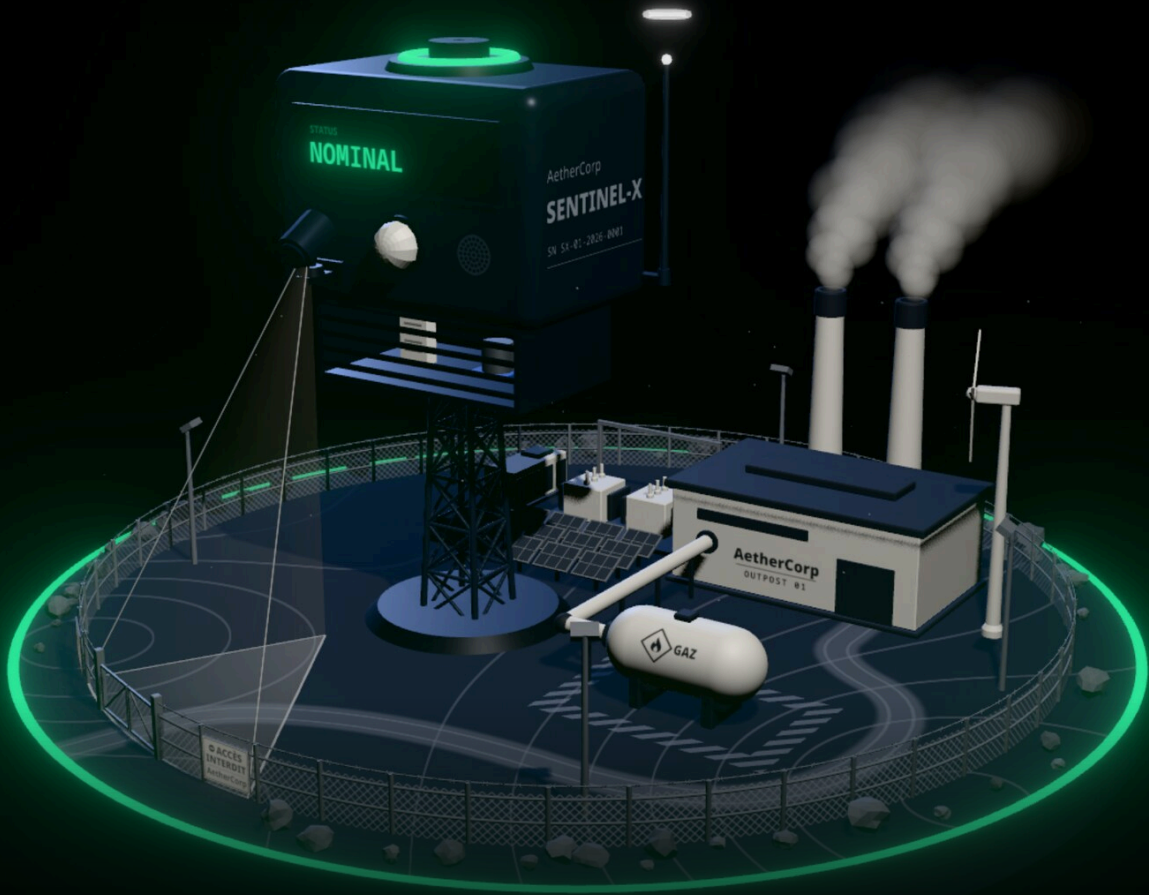




SENTINEL - X

Dossier d'ingénierie technique

Groupe G2 · Option A, centralisation embarquée sur Raspberry Pi

CONTENU

1	Schéma réseau
2	Schéma de câblage électronique
3	Matrice de sécurité
4	Documentation de l'IA
5	Rapport d'audit post - pentest
Annexe	Poster A3

ÉQUIPE

DEV	Arthur · API, jumeau numérique, services IA
DEV	Amen · Dashboard, vue Opérateur
DEV	Nathan · Firmware ESP32, installation du Pi
INFRA	Safi · Infrastructure, réseau, cybersécurité
INFRA	Florent · Infrastructure, réseau, cybersécurité

0 Le système en bref

Sentinel-X protège une micro-centrale isolée contre trois menaces : gaz et surchauffe, intrusion physique, attaque réseau. Un seul boîtier réunit le **Sentinel** (ESP32 et ses sondes) et le **Command Post** (Raspberry Pi 4, le « PC Serveur Local » du sujet). Le PC de l'Opérateur n'apporte qu'un navigateur.

1 s entre deux mesures envoyées par le Sentinel	2 ports de service ouverts à la table : 443 et 8883	TLS 1.2+ sur tout flux qui quitte une machine	6 conteneurs, tous non root et en lecture seule	1 483 tests automatisés, rejoués à chaque pull request
--	---	--	--	--

Réponse au sujet

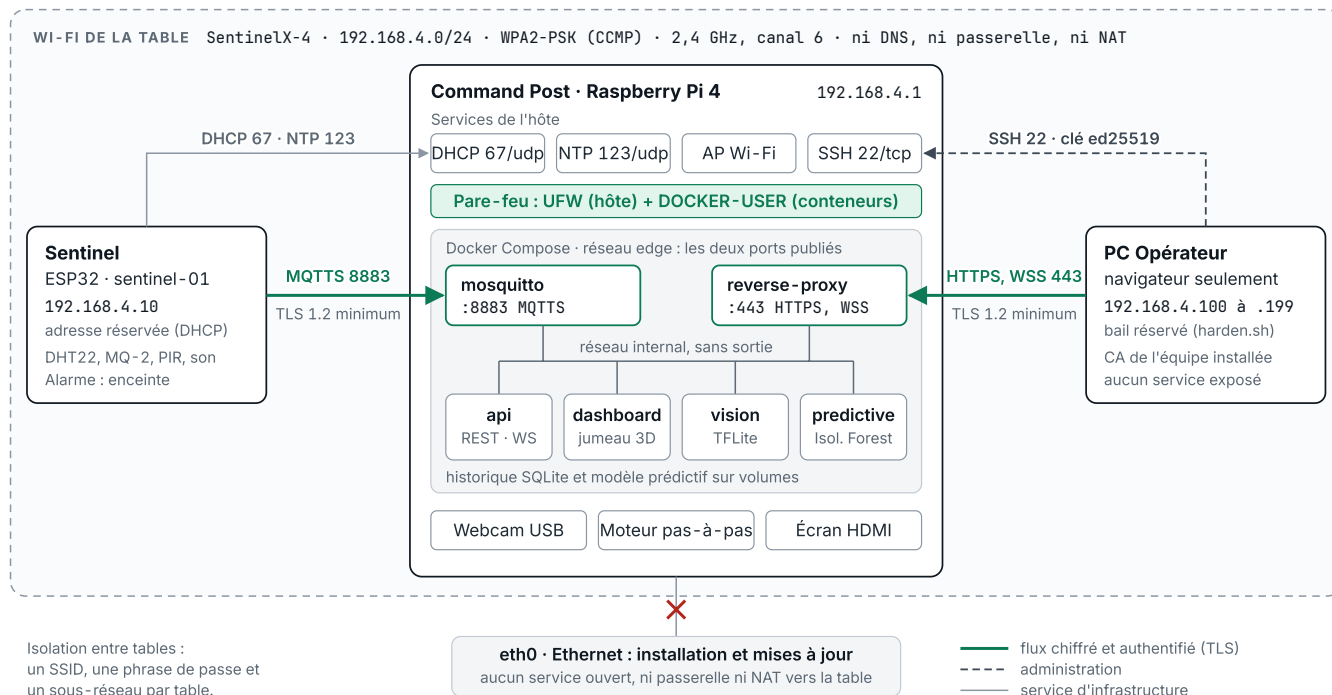
EXIGENCE DU SUJET	RÉPONSE DE L'ÉQUIPE	OÙ
DEV · CONCEPTION ET DÉVELOPPEMENT		
Micrologiciel C++, capteurs cadencés, payloads	Firmware C++ (PlatformIO) : un instantané JSON par seconde, Alertes à hystérésis, Alarme autonome	firmware/
Affichage local du statut	Écran HDMI du Command Post (écart 3)	infra/screen.py
API REST ou WebSocket	API Node.js (Fastify) : REST, WebSocket, POST /api/v1/alerts	backend/
Dashboard : courbes, statut, retour webcam	React : jumeau 3D, courbes, Status, caméra, rejeu d'incident	dashboard/
Commande à distance des actionneurs	Commande de l'Alarme : API, puis MQTTS, puis ESP32	backend/
IA · INTELLIGENCE ARTIFICIELLE ET DATA		
Vision : présence humaine en temps réel	Service vision : EfficientDet-Lite0 (TFLite), classe personne	section 4
Maintenance prédictive sans seuil statique	Service predictive : Isolation Forest, niveaux appris	section 4
Images 640 × 480, moins de 100 ms par trame	Capture en 640 × 480, modèle en 320 × 320, banc de mesure	section 4
INFRA · RÉSEAU ET SYSTÈMES		
Stack Docker-Compose : base, API, broker	6 services, redémarrage automatique, historique SQLite	docker-compose.yml
Point d'accès Wi-Fi dédié, adressage, isolation	Sous-réseau propre à la table, sans route vers l'extérieur	section 1
Monitoring et MCO	Écran de statut, santé Docker, rotation des logs, docker stats	infra/
CYBER · CYBERSÉCURITÉ		
Chiffrement de bout en bout, de l'ESP à la stack	MQTTS (TLS 1.2 minimum), CA de l'équipe ; HTTPS côté Opérateur	section 3
Durcissement du serveur	SSH par clé, pare-feu, ports Docker filtrés, services coupés	section 3
Audit de sécurité et pentest croisé	Grille de 19 tests, audit d'une table consentante	section 5

Écarts au sujet

ÉCART	JUSTIFICATION
1 ESP32 à la place de l'ESP8266	Même famille, même chaîne d'outils (Arduino, PlatformIO). Assez de mémoire pour le TLS obligatoire, et plusieurs entrées analogiques contre une seule, limitée à 1 V.
2 Raspberry Pi 4 à la place du Pi 5	Matériel disponible. Inférence plus lente : modèle léger et leviers de la section 4.
3 Écran HDMI sur le Pi à la place de l'OLED sur le microcontrôleur	Même statut IP et Wi-Fi, plus l'état du Sentinel, ses mesures, les Alertes et les conteneurs.

1 Schéma réseau

Le Command Post est le point d'accès d'un sous-réseau Wi-Fi isolé, propre à la table. Rien ne route vers un autre réseau : ni passerelle, ni DNS, ni NAT.



PLAN D'ADRESSAGE · MASQUE 255.255.255.0 · BAIL DHCP DE 12 H

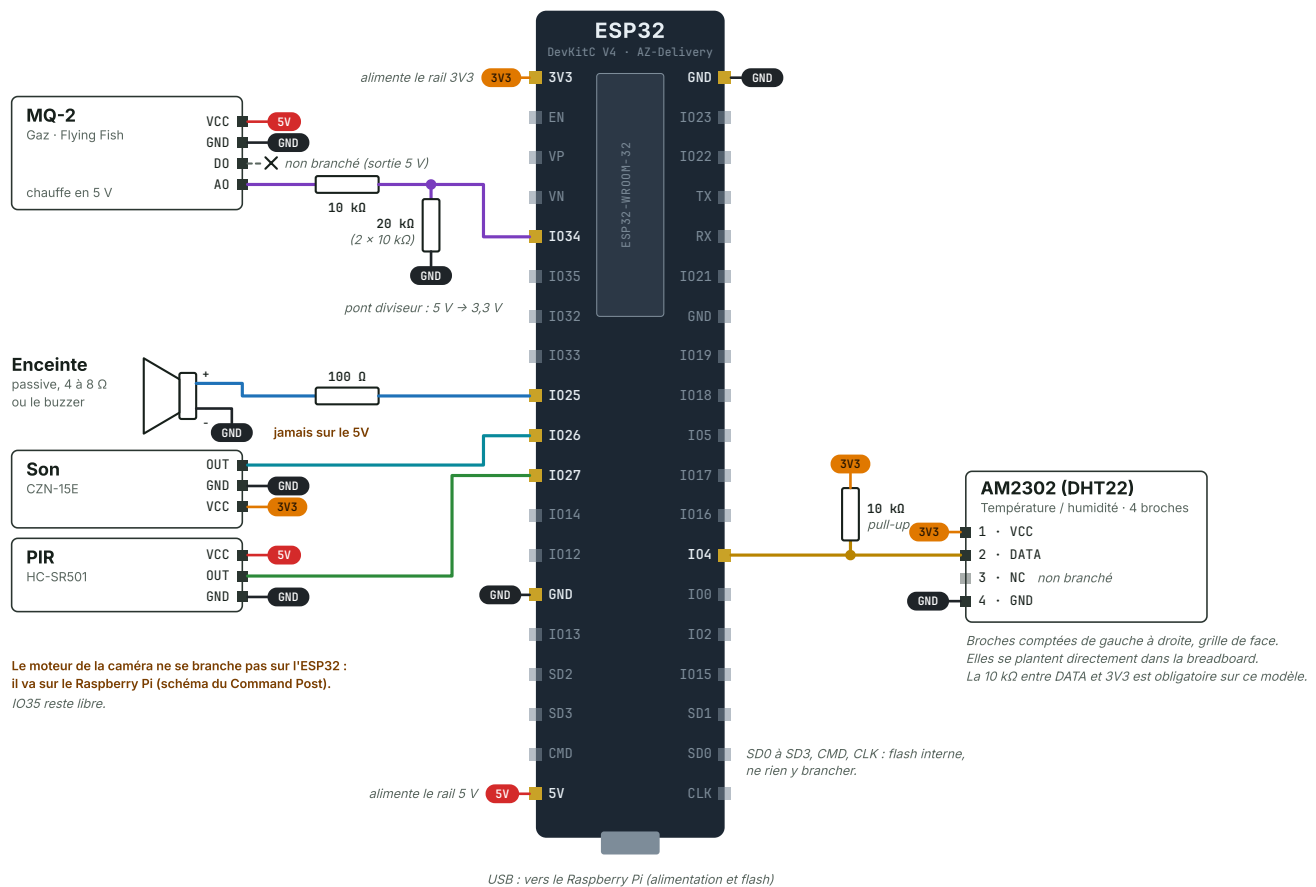
HÔTE	ADRESSE	ATTRIBUTION	RÔLE
Command Post (Raspberry Pi 4)	192.168.4.1	statique, interface wlan0	point d'accès, DHCP, heure, stack Docker
Sentinel (ESP32, sentinel-01)	192.168.4.10	réservée par nom d'hôte	sondes et Alarme
PC Opérateur	192.168.4.100 à .199	DHCP, bail réservé par adresse MAC	navigateur, SSH d'administration

MATRICE DES FLUX

DE	VERS	PROTOCOLE, PORT	PROTECTION	CONTENU
Sentinel	mosquitto	MQTTS 8883/tcp	TLS 1.2 minimum, CA de l'équipe vérifiée par l'ESP32 ; compte sentinel-01 et ACL	télémétrie et Alertes ; en retour, commandes de l'Alarme
PC Opérateur	reverse-proxy	HTTPS, WSS 443/tcp	TLS 1.2 minimum, CA de l'équipe ; session Opérateur	dashboard, API, temps réel, caméra
PC Opérateur	hôte (sshd)	SSH 22/tcp	clé ed25519, un seul compte, cette seule adresse	administration
api, predictive, écran de statut	mosquitto	MQTTS 8883/tcp	TLS, CA vérifiée ; un compte et une ACL par client	lecture ; l'api publie les commandes
vision, predictive	api	HTTP 8080, interne	réseau Docker sans sortie, même hôte ; un jeton par service	POST /api/v1/alerts
reverse-proxy	api, dashboard, vision	HTTP, interne	session vérifiée par l'api avant le flux caméra	routage de /api, /ws, /camera
Sentinel, PC Opérateur	hôte	DHCP 67/udp, NTP 123/udp	services d'infrastructure, sans donnée métier	adresses, heure qui date chaque mesure

2 Schéma de câblage électronique

Le Sentinel est un ESP32 DevKitC V4 (38 broches) qui porte les sondes et l'Alarme. La webcam, son moteur et l'écran sont reliés au Raspberry Pi 4. Les broches sont celles de `firmware/include/pins.h`.

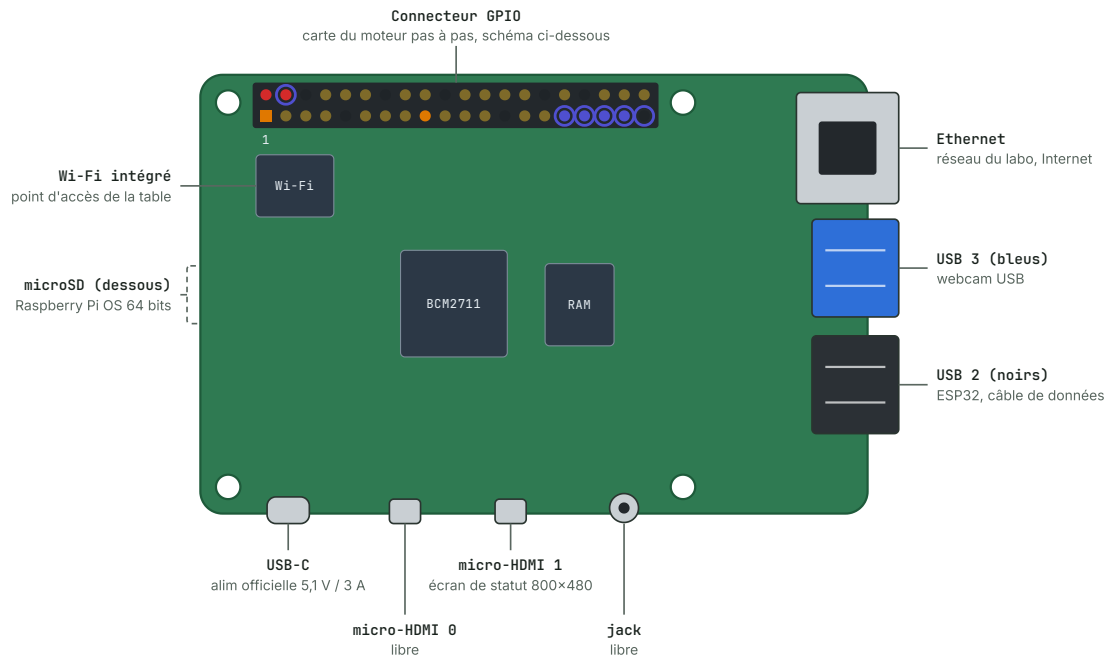


Le Sentinel sur breadboard, sans soudure. Fiche de câblage de l'équipe : [docs/cablage/esp32.html](https://docs.cablage/esp32.html).

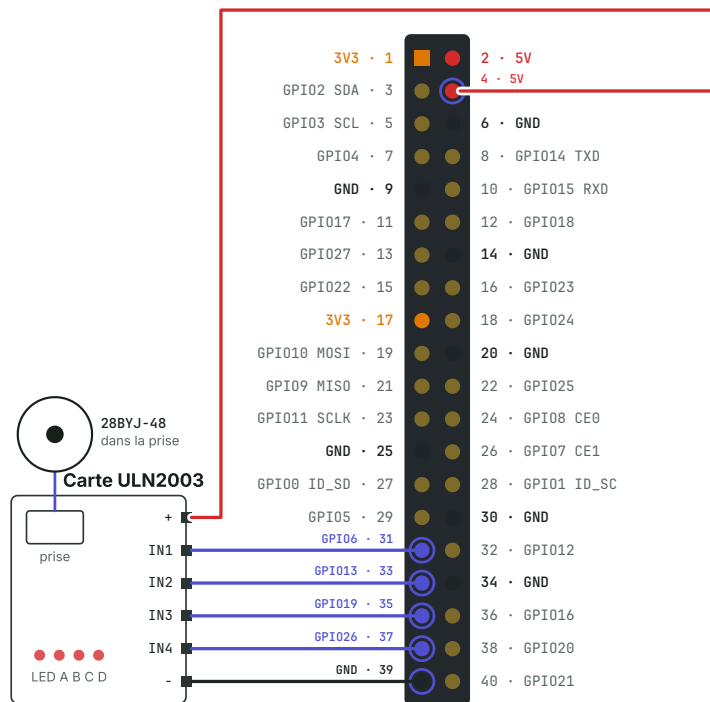
SENTINEL · ESP32

COMPOSANT	RÔLE	SIGNAL	MONTAGE
DHT22	température, humidité	DATA sur GPIO 4	résistance de tirage de 10 kΩ vers 3V3
MQ-2	gaz, fumées	AO sur GPIO 34 (ADC1)	chauffage en 5 V ; pont diviseur 10 kΩ / 20 kΩ pour que la sortie reste sous 3,3 V
PIR HC-SR501	présence	OUT sur GPIO 27	sortie déjà en 3,3 V
CZN-15E	niveau sonore	DO sur GPIO 26	sortie numérique, seuil réglé au potentiomètre du module
Enceinte 4 à 8 Ω	Alarme (sirène)	GPIO 25	entre la broche et la masse, à travers 100 Ω : environ 30 mA, ce qu'une broche peut fournir ; jamais sur le 5 V

Les sondes analogiques sont sur l'ADC1 : l'ADC2 est indisponible quand le Wi-Fi est actif.



Ce qui se branche sur chaque port du Raspberry Pi 4.



Pi tourné d'un quart de tour : connecteur à droite, ports USB en bas. Broche 1 : pastille carrée, à l'opposé des ports USB.

Le moteur pas-à-pas de la caméra sur le connecteur GPIO. Fiche de câblage de l'équipe : <docs/cablage/raspberry-pi.html>.

COMMAND POST · RASPBERRY PI 4

PÉRIPHÉRIQUE	LIAISON	RÔLE
Webcam USB	port USB	image du service vision
Moteur pas-à-pas 28BYJ-48 (unipolaire, 5 fils) et carte driver ULN2003	entrées IN1 à IN4 de la carte sur GPIO 6, 13, 19 et 26 (broches 31, 33, 35 et 37) ; IN5 à IN7 non utilisées ; carte alimentée par le 5 V du Pi (broche 4, masse en broche 39), environ 200 mA	oriente la webcam vers la personne suivie
Écran HDMI 800 × 480	micro-HDMI	écran de statut
ESP32	USB	flash du firmware, moniteur série
Alimentation	bloc USB-C officiel 5,1 V / 3 A	Raspberry Pi 4

3 Matrice de sécurité

Tout flux qui quitte une machine est chiffré et authentifié, et le serveur n'expose que ce dont la table a besoin. La dernière colonne renvoie aux tests D1 à D19 de la grille d'audit (section 5).

MESURE	MISE EN ŒUVRE	TEST
CHIFFREMENT DES FLUX (TLS)		
MQTT chiffré seulement	Un seul listener, MQTTS sur 8883, <code>tls_version</code> <code>tlsv1.2</code> ; aucun port 1883	D5, D6, D8
HTTPS seulement	Caddy, TLS 1.2 minimum, en-tête HSTS ; aucun port 80	D5, D7, D8
CA de l'équipe	Autorité de certification ECDSA P-256 créée sur le Pi (<code>infra/setup.sh</code>). Certificats : broker (IP:192.168.4.1, DNS:mosquitto) et proxy (adresse du Pi, <code>sentinel-x.local</code>)	D6, D7
Serveur vérifié par le client	ESP32 : <code>setCACert</code> , jamais <code>setInsecure()</code> . Services et écran de statut : CA imposée. PC Opérateur : CA installée par <code>infra/operator.sh</code>	D6, étape 5
AUTHENTIFICATION ET AUTORISATIONS		
Broker fermé aux anonymes	<code>allow_anonymous</code> <code>false</code> ; un compte par client (<code>sentinel-01</code> , <code>api</code> , <code>predictive</code> , <code>screen</code>), mots de passe aléatoires de 192 bits, fichier haché	D9
Moindre privilège MQTT	ACL par compte : le Sentinel n'écrit que sa télémétrie et ses Alertes, ne lit que ses commandes	D10
Services IA	Un jeton de 256 bits par service, limité à son type d'Alerte ; l'api fixe elle-même la source	D12, D13
Session Opérateur	Mot de passe de 12 caractères ou plus, haché <code>bcrypt</code> ; cookie <code>HttpOnly</code> ; <code>Secure</code> ; <code>SameSite=Strict</code> de 12 h ; origine vérifiée sur le WebSocket	D11, D19
Entrées non fiables	Chaque corps validé contre le schéma (champs inconnus refusés, 16 Ko au plus) ; l'identité du Sentinel vient du topic, pas du message	D13
DURCISSEMENT DU SERVEUR (HARDENING) · CYBER/HARDEN.SH		
SSH	Clé <code>ed25519</code> seulement ; mot de passe et <code>root</code> refusés ; un seul compte ; depuis le PC Opérateur seulement	D4
Pare-feu de l'hôte	UFW : tout entrant refusé, sauf sur <code>wlan0</code> 22/tcp (PC Opérateur), 67/udp et 123/udp	D1, D2
Ports publiés par Docker	Docker contourne UFW : la chaîne <code>SENTINEL-X</code> , appelée depuis <code>DOCKER-USER</code> , n'ouvre 443 et 8883 qu'au Wi-Fi de la table, et une unité <code>systemd</code> la remet à chaque démarrage	D3
Services inutiles	<code>bluetooth</code> , <code>hciuart</code> , <code>avahi-daemon</code> , <code>triggerhappy</code> , <code>ModemManager</code> coupés	checklist
Wi-Fi de la table	WPA2-PSK, CCMP seulement, phrase de passe aléatoire de 24 caractères ; ni passerelle ni NAT	D18
ISOLATION DES CONTENEURS DOCKER		
Privilèges	Chaque conteneur : utilisateur non <code>root</code> , lecture seule, <code>cap_drop: ALL</code> , <code>no-new-privileges</code> . Aucun <code>privileged</code> , socket Docker jamais monté	D17
Réseau	<code>api</code> , <code>dashboard</code> , <code>vision</code> , <code>predictive</code> sur un réseau Docker interne sans sortie ; seuls le proxy et le broker publient un port	D1, D17
Périphériques	La webcam est passée à <code>vision</code> par son seul nœud <code>/dev</code> , par le groupe <code>video</code>	D17
DÉNI DE SERVICE ET SECRETS		
Limites de débit	Connexion : 5 par minute et par adresse. Commandes : 2 par seconde. Alertes : 20 par seconde et par jeton. Au-delà : 429	D14, D15
Inondation MQTT	<code>max_packet_size</code> 4096, broker sans persistance, logs en rotation (10 Mo × 3 par conteneur) : la carte SD ne se remplit pas	D16
Secrets hors du dépôt	Générés sur le Pi dans <code>infra/secrets/</code> (ignoré par git, droits 600). La CI refuse tout fichier secret suivi	CI

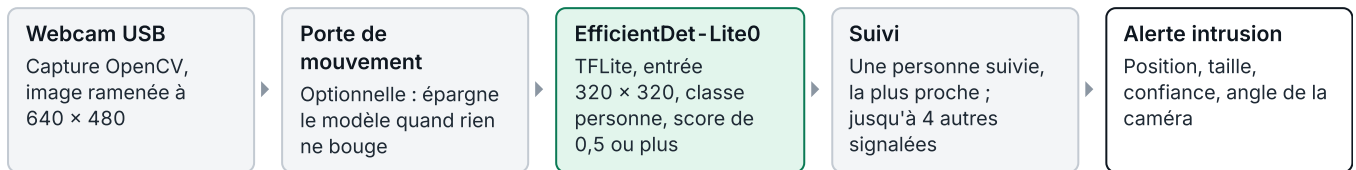
Preuves du durcissement relevées sur le Command Post, depuis le PC Opérateur

PREUVE	COMMANDE	ATTENDU	ÉTAT
Ports TCP	<code>nmap -sS -p- 192.168.4.1</code>	22, 443 et 8883 ouverts, les 65 532 autres filtrés	à relever
Ports UDP	<code>nmap -sU -p 53,67,68,69,123,137,161,1900,5353 192.168.4.1</code>	123 ouvert, 67 confirmé par dhcp-discover, aucun autre service	à relever
Checklist	<code>cyber/harden.sh 4 --check</code>	toutes les lignes cochées, y compris après redémarrage	à relever

4 Documentation de l'IA

Deux services Python en conteneur sur le Command Post. Chacun publie ses Alertes par POST /api/v1/alerts avec son propre jeton ; 437 tests automatisés les couvrent, sans matériel.

4.1 Vision : détection d'intrusion service vision · ai/vision/



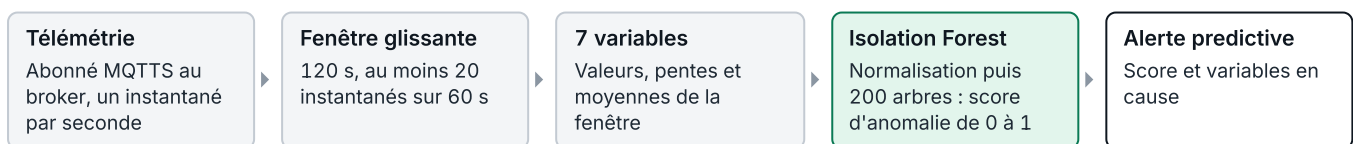
Modèle	EfficientDet-Lite0 (COCO, Apache 2.0), adapté de automaticai/rpi-object-detection (MIT), TensorFlow Lite sur 4 threads. Poids téléchargés à la construction de l'image, sha256 vérifié.
Cycle de l'Alerte	Levée après 2 inférences de suite avec une personne (sévérité critique), mise à jour au plus toutes les 500 ms sous le même identifiant, fin après 3 s sans personne.
Suivi par la caméra	Le moteur pas-à-pas garde la personne suivie au centre de l'image ; sans intrusion, il balaie 30° de chaque côté. L'angle part dans chaque Alerte et le jumeau numérique place l'intrus sur le site.
Retour visuel, repli	Flux MJPEG annoté à 15 images par seconde. Repli autorisé par le sujet : mouvement OpenCV seul (DETECTOR=motion).

PERFORMANCE · EXIGENCE : 640 × 480, MOINS DE 100 MS PAR TRAME · OUTIL : PYTHON -M VISION.BENCH

MACHINE	DÉTECTEUR	INFÉRENCE P50 / P95 (MS PAR IMAGE)
PC de développement (4 vCPU)	TFLite, 4 threads	10,6 / 13,1
	TFLite, 4 threads, porte de mouvement	12,2 / 15,0
	Mouvement seul (repli)	2,1 / 3,0
Raspberry Pi 4	TFLite, 4 threads	à relever
Raspberry Pi 4	TFLite, 4 threads, porte de mouvement	à relever

Leviers si le Pi 4 dépasse 100 ms : une inférence toutes les deux ou trois images (INFER_EVERY), porte de mouvement, repli OpenCV.

4.2 Maintenance prédictive service predictive · ai/predictive/



VARIABLES	SENS	RÈGLE	VALEUR
temp, humidity, air	dernières mesures	Niveaux appris	levée au quantile 0,999, fin au quantile 0,99 des scores d'un nominal tenu à l'écart de l'entraînement
temp_slope, air_slope	tendance par minute (moindres carrés)	Hystérésis	levée après 5 scores de suite au-dessus, fin après 10 de suite en dessous
temp_mean, air_mean	moyennes sur la fenêtre		

Aucun seuil statique. Le modèle apprend le nominal seul, lu dans l'historique de l'api ; les périodes sous Alerte sont écartées, l'entraînement est refusé sous 100 vecteurs.

VALIDATION

JEU DE DONNÉES	CONTENU	VECTEURS	LEVÉE / FIN	RÉSULTAT
Simulation predictive simulate	2 h de nominal synthétique, puis dérive lente chaleur et gaz	7 140	0,602 / 0,580	Alerte 2 min 47 s après le début de la dérive : 43 min 19 s avant le seuil thermique du firmware, 47 min 17 s avant son seuil gaz
Capture réelle (Sentinel)	à relever			

5 Rapport d'audit post-pentest

Pentest croisé du jeudi : nous vérifions notre Command Post avec une grille de 19 tests, puis nous auditons une table consentante avec la même méthode.

DATE	jeudi 8 octobre 2026
AUDITEURS	à relever
TABLE AUDITÉE	à relever
AUDITÉS PAR	à relever
OUTILS	Nmap, Wireshark, Metasploit, openssl s_client, mosquito_pub et mosquito_sub, curl
RÈGLES	Tables consentantes du workshop seulement. Ni destruction ni persistance. Déni de service court et annoncé. Aucun secret conservé.

5.1 Synthèse

à relever

5.2 Notre Command Post : grille défensive

	TEST	ATTENDU	OBSERVÉ	VERDICT
D1	Ports TCP exposés (nmap -sS -p-)	22, 443 et 8883 seuls, le reste filtré		à relever
D2	Ports UDP (nmap -sU, scripts dhcp-discover et ntp-info)	67 et 123 seuls		à relever
D3	Docker ne contourne pas le pare-feu (scan depuis l'Ethernet)	443 et 8883 muets hors du Wi-Fi de la table		à relever
D4	SSH par mot de passe	refusé, clé ed25519 seule, depuis le PC Opérateur		à relever
D5	Ports en clair (MQTT 1883, HTTP 80)	aucun, connexion refusée		à relever
D6	TLS du broker (openssl s_client sur 8883)	TLS 1.2 ou plus, certificat de la CA d'équipe, échec sans la CA		à relever
D7	TLS du dashboard (openssl s_client sur 443)	HTTPS seul, certificat de la CA d'équipe		à relever
D8	Capture Wireshark d'une session de démo	ni identifiant ni mesure lisible		à relever
D9	Broker sans identifiants	connexion refusée		à relever
D10	ACL : sentinel-01 publie sur command/...	refusé, chaque compte reste dans son périmètre		à relever
D11	API, WebSocket et caméra sans session	401		à relever
D12	Jeton vision avec un autre type d'Alerte	403		à relever
D13	Alerte au champ sentinel ou source mensonger	ignoré, valeurs fixées par l'api		à relever
D14	Plus de 5 tentatives de connexion par minute	429 avec Retry-After		à relever
D15	Plus de 2 commandes/s, plus de 20 Alertes/s	rejet au-delà de la limite		à relever
D16	Message MQTT de plus de 4096 octets, rafale	message rejeté, service maintenu		à relever

	TEST	ATTENDU	OBSERVÉ	VERDICT
D17	Revue des conteneurs	aucun privilège superflu, socket Docker non monté		à relever
D18	Revue du point d'accès Wi-Fi	WPA2-PSK, CCMP seul, WPS coupé, pas de NAT		à relever
D19	Cookie de session et origine du WebSocket	HttpOnly, Secure, SameSite=Strict ; autre origine : 403		à relever

5.3 Audit offensif d'une table consentante

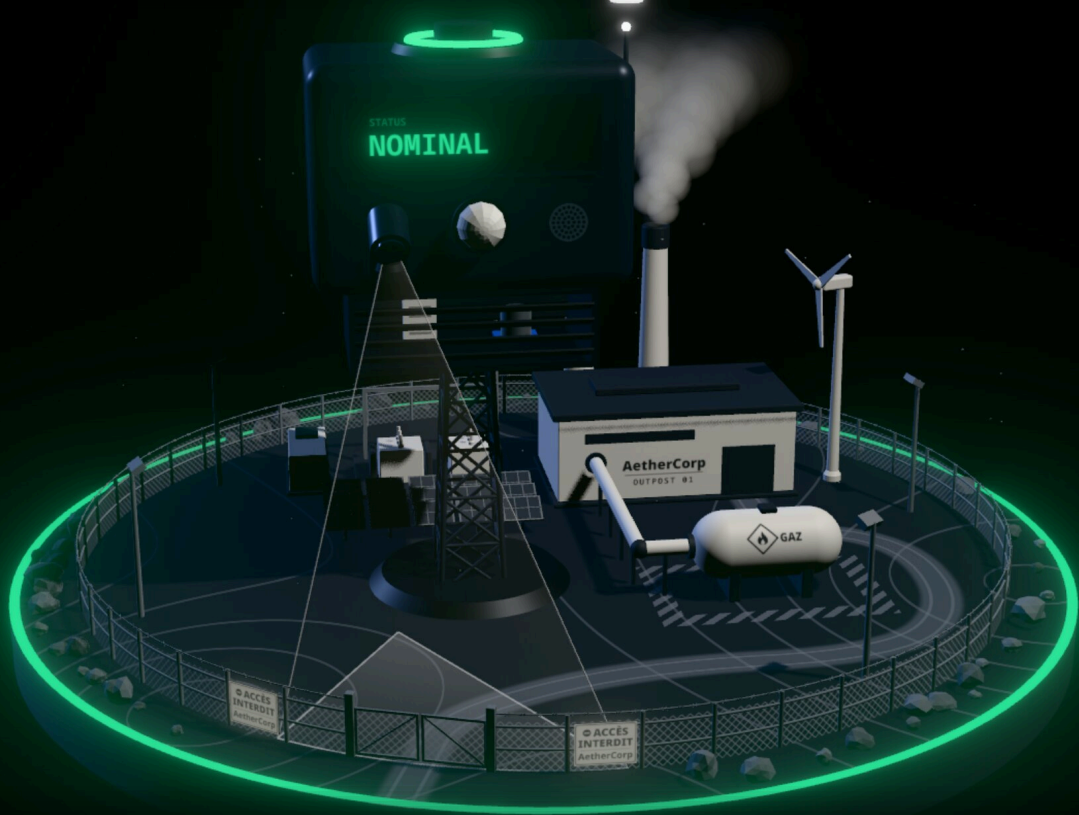
	ÉTAPE	CE QUI A ÉTÉ TENTÉ	RÉSULTAT
1	Reconnaissance (Nmap)	à relever	
2	Transport (TLS, Wireshark)	à relever	
3	Authentification (broker, API)	à relever	
4	Résilience (DoS court, cible prévenue)	à relever	
5	Homme du milieu (avec accord)	à relever	
6	Restitution à la table auditée	à relever	

5.4 Constats et correctifs

SYSTÈME	CRITICITÉ	CONSTAT	CORRECTIF	ÉTAT
à relever				

5.5 Conclusion

à relever



SENTINEL - X

La sécurité à la bordure.

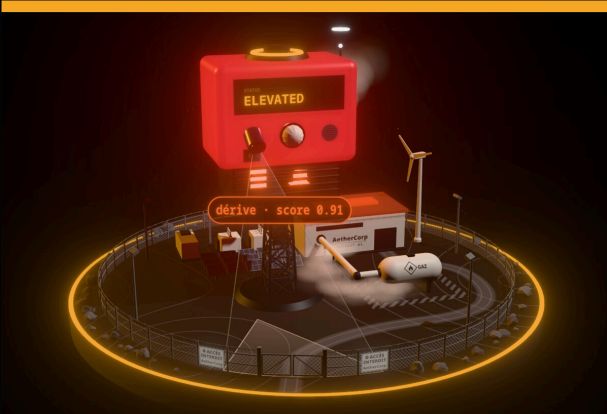
Un boîtier autonome qui surveille, comprend et défend une micro-centrale isolée, là où personne ne peut veiller.



DÉTECTER

Fuite de gaz, surchauffe

Les sondes sont lues chaque seconde. Au seuil critique, la sirène sonne sur place, même serveur coupé.



ANTICIPER

La dérive avant l'incident

Un modèle Isolation Forest apprend le fonctionnement normal et signale une dérive lente bien avant le seuil.



REPÉRER

Intrusion physique

La caméra reconnaît une personne, la suit, et le jumeau numérique 3D la place sur le site en direct.

PROTÉGER

Une chaîne chiffrée de la sonde à l'écran

Sentinel

ESP32, sondes gaz, température, présence et son, sirène

MQTTs

TLS 1.2 minimum

Command Post

Raspberry Pi 4 dans le boîtier : broker, API, IA et dashboard dans 6 conteneurs sans privilège

HTTPS, WSS

session Opérateur

Opérateur

Jumeau numérique 3D, courbes, caméra, commande de l'alarme

1 s

entre deux mesures du Sentinel

2

ports de service ouverts, rien d'autre ne répond

TLS 1.2+

et une autorité de certification propre à l'équipe

7

variables pour le modèle prédictif, aucun seuil fixe

1 483

tests automatisés, rejoués à chaque pull request